

DOCUMENTACIÓN TÉCNICA Y FUNCIONAL DEL SISTEMA OMNIMERGE

Plataforma Web de Creación, Organización, Reutilización y Simulación de Universos, Entidades y Torneos

Nombre del sistema: OmniMerge

Tipo de sistema: Plataforma web multiusuario

Arquitectura: Modelo–Vista–Controlador, MVC

Framework principal: Laravel

Base de datos: MySQL

Estado del documento: Documentación del desarrollo realizado hasta el módulo de exploración comunitaria

Fecha de actualización: 5 de agosto de 2026

Rama principal: [main](#)

Repositorio: [grverlearner/OmniMerge](#)

1. PRESENTACIÓN DEL PROYECTO

OmniMerge es una plataforma web diseñada para permitir que diferentes usuarios creen, organicen, personalicen y reutilicen entidades de cualquier tipo.

Una entidad no está limitada a ser una persona o personaje humanoide. En OmniMerge una entidad puede representar:

- Un personaje.
- Un animal.
- Un país.
- Un objeto.
- Un vehículo.
- Una criatura.
- Un planeta.
- Un color.
- Una habilidad.
- Una organización.
- Un concepto abstracto.
- Cualquier otro elemento que el usuario desee modelar.

El sistema permite que cada usuario defina sus propios tipos de entidad, atributos, valores seleccionables, grupos de atributos y colecciones. Esto evita trabajar con una estructura rígida y convierte a OmniMerge en una plataforma flexible y extensible.

La etapa desarrollada hasta el momento se concentra en la construcción de la biblioteca de contenido de OmniMerge. Esta biblioteca será la base para módulos posteriores como universos, temporadas, torneos, simulaciones, rankings e historias emergentes.

2. SITUACIÓN DEL REPOSITORIO

El repositorio público de OmniMerge utiliza la rama [main](#) y conserva la estructura principal de una aplicación Laravel: [app](#), [bootstrap](#), [config](#), [database](#), [public](#), [resources](#), [routes](#), [storage](#) y [tests](#). El repositorio también contiene migraciones correspondientes a usuarios, tipos de entidad, entidades, atributos, grupos, opciones, valores, colecciones y comunidad.

El archivo [README.md](#) público todavía conserva principalmente la documentación genérica de Laravel, por lo que el presente documento está preparado para reemplazarlo o utilizarse como documentación técnica independiente.

La documentación refleja:

1. La planificación funcional realizada durante el desarrollo.
2. El código y la arquitectura trabajados durante los distintos sprints.
3. Las migraciones visibles en la rama [main](#).
4. Los módulos probados localmente.
5. Las mejoras y módulos pendientes.

Es posible que algunos cambios locales todavía no se encuentren publicados mediante [git push](#). Por ello, antes de entregar el proyecto se recomienda comparar esta documentación con el resultado de:

```
git status
```

```
git log --oneline
```

```
php artisan route:list
```

```
php artisan migrate:status
```

3. DESCRIPCIÓN DEL PROBLEMA

Actualmente, la mayoría de herramientas para la creación de personajes, historias, simulaciones o competencias presentan una o varias de las siguientes limitaciones:

- Están limitadas a personajes humanoides.
- Poseen formularios rígidos con características predefinidas.
- Están orientadas a un videojuego específico.
- No permiten crear atributos personalizados.
- No permiten que un atributo posea un catálogo propio.
- No permiten asignar varios valores al mismo atributo.
- No ofrecen una estructura reutilizable entre diferentes proyectos.
- No permiten organizar entidades en colecciones.
- No mantienen una evolución histórica de las entidades.
- No permiten explorar y copiar contenido creado por otros usuarios.
- No permiten simular torneos automáticos basados en atributos.
- Requieren intervención directa durante el desarrollo de la simulación.

Como consecuencia, el usuario no dispone de un espacio flexible donde pueda crear cualquier elemento, describirlo mediante características propias, reutilizarlo en distintos contextos y observar su comportamiento dentro de universos o competencias automáticas.

4. PROPUESTA DE SOLUCIÓN

OmniMerge propone una plataforma web multiusuario donde cada persona pueda construir su propia biblioteca de contenido.

La solución se organiza alrededor de los siguientes conceptos:

4.1. Tipos de entidad

Permiten clasificar las entidades.

Ejemplos:

- Personaje.
- País.
- Animal.
- Planeta.
- Objeto.
- Criatura.
- Organización.
- Concepto.

4.2. Entidades

Representan los elementos concretos creados por el usuario.

Ejemplos:

- Naruto Uzumaki.
- Perú.
- Dragón de fuego.
- Espada legendaria.
- Planeta Tierra.
- Color rojo.

4.3. Atributos

Representan las características que pueden asignarse a una entidad.

Ejemplos:

- Anime.
- Elemento.
- Poder.
- Edad.
- País de origen.
- Puede volar.
- Fecha de aparición.
- Historia.
- Color principal.

4.4. Opciones de atributo

Representan los valores seleccionables de un atributo basado en catálogo.

Ejemplo:

Atributo: Anime

Opciones:

- Naruto
- One Piece
- Dragon Ball
- Bleach

Otro ejemplo:

Atributo: Elemento

Opciones:

- Fuego
- Agua
- Tierra
- Aire
- Luz
- Oscuridad

Las opciones pueden contener:

- Nombre.
- Código.
- Descripción.
- Imagen.
- Icono.
- Color.
- Valor numérico.
- Opción superior.
- Orden.
- Estado.

4.5. Grupos de atributos

Permiten organizar visualmente los atributos.

Ejemplos:

- Información general.
- Apariencia.
- Personalidad.
- Combate.
- Poderes.
- Historia.
- Datos geográficos.

4.6. Colecciones

Permiten agrupar entidades de manera reutilizable.

Ejemplos:

- Protagonistas de anime.
- Países de América del Sur.
- Dragones elementales.
- Objetos legendarios.
- Participantes del torneo.
- Personajes favoritos.

4.7. Biblioteca comunitaria

Permite explorar contenido público creado por otros usuarios y guardar una copia independiente en la biblioteca personal.

El contenido de otra persona no se modifica directamente. Se clona para que el usuario pueda editar su copia sin afectar el registro original.

5. JUSTIFICACIÓN

5.1. Justificación técnica

OmniMerge utiliza una arquitectura flexible basada en entidades, atributos dinámicos y valores tipados.

Esta arquitectura permite:

- Crear estructuras personalizadas.
- Evitar columnas fijas para cada característica.
- Añadir nuevos tipos de dato.
- Reutilizar atributos.
- Crear catálogos seleccionables.
- Asignar múltiples valores.
- Escalar hacia filtros y rankings.
- Implementar condiciones y jerarquías.
- Incorporar simulaciones en futuras versiones.

5.2. Justificación operativa

El sistema centraliza en una sola plataforma:

- Creación de entidades.
- Clasificación.
- Administración de atributos.
- Administración de catálogos.
- Agrupación en colecciones.
- Publicación comunitaria.
- Clonación de contenido.
- Preparación de universos.
- Preparación de torneos.

5.3. Justificación académica

OmniMerge puede utilizarse para estudiar:

- Diseño de bases de datos flexibles.
- Arquitectura MVC.
- Desarrollo web con Laravel.
- Relaciones Eloquent.
- Sistemas multiusuario.
- Modelado dinámico de información.
- Simulación basada en reglas.
- Reutilización de conocimiento.
- Diseño de sistemas creativos.

5.4. Justificación creativa

El usuario puede construir contenido sin estar limitado por una temática determinada.

Una misma plataforma puede utilizarse para:

- Anime.
- Fantasía.
- Historia.
- Geografía.
- Ciencia ficción.

- Educación.
 - Deportes.
 - Objetos.
 - Animales.
 - Juegos narrativos.
 - Simulaciones experimentales.
-

6. OBJETIVOS

6.1. Objetivo general

Desarrollar una plataforma web que permita a los usuarios crear entidades personalizadas, organizarlas en universos independientes y simular torneos automáticos basados en atributos y reglas definidas, generando evolución histórica, estadísticas y narrativa emergente.

6.2. Objetivos específicos

1. Permitir la creación, edición y reutilización de entidades genéricas.
 2. Gestionar tipos de entidad personalizados.
 3. Gestionar atributos dinámicos.
 4. Permitir atributos de texto, números, fechas, colores, booleanos y catálogos.
 5. Permitir que un atributo tenga una o varias opciones seleccionables.
 6. Permitir imágenes en entidades, colecciones, atributos y opciones.
 7. Organizar atributos mediante grupos visuales.
 8. Organizar entidades mediante colecciones.
 9. Implementar seguridad por usuario.
 10. Permitir contenido privado, público o no listado.
 11. Permitir explorar contenido público.
 12. Permitir clonar contenido comunitario.
 13. Preparar la plataforma para universos.
 14. Preparar la plataforma para temporadas.
 15. Preparar la plataforma para torneos automáticos.
 16. Registrar resultados, estadísticas e historial.
 17. Facilitar rankings y análisis posteriores.
-

7. ALCANCE ACTUAL

Actualmente se ha trabajado en los siguientes módulos:

- Configuración inicial de Laravel.
- Conexión con MySQL.
- Control de versiones con Git y GitHub.
- Registro de usuarios.
- Inicio y cierre de sesión.
- Perfil de usuario.
- Dashboard.
- Menú lateral.
- Tipos de entidad.
- Entidades.
- Carga de imágenes.

- Atributos dinámicos.
- Opciones de atributos.
- Valores múltiples.
- Grupos de atributos.
- Colecciones.
- Explorador comunitario.
- Clonación de entidades.
- Clonación de atributos.
- Clonación de colecciones.
- Registro de vistas y clonaciones.

Los siguientes módulos todavía forman parte de la evolución futura:

- Universos.
- Temporadas.
- Torneos.
- Simulación automática.
- Motor de reglas.
- Relaciones avanzadas entre atributos.
- Condiciones de visibilidad.
- Rankings.
- Historial de eventos.
- Favoritos comunitarios.
- Reportes.
- Moderación.
- Notificaciones.
- API pública.

8. LIMITACIONES ACTUALES

- No existe todavía una simulación de combate o competencia.
- No se utilizan modelos avanzados de inteligencia artificial.
- No se incluyen gráficos 3D.
- No existen pagos ni monetización.
- No existe todavía moderación comunitaria.
- No existe un sistema completo de seguidores.
- No se han implementado notificaciones.
- Las jerarquías entre atributos todavía necesitan un módulo independiente.
- La taxonomía de tipos de entidad es creada por cada usuario.
- Los tests originales de Breeze necesitan adaptarse completamente a [username](#).
- Algunos cambios pueden encontrarse localmente y no en GitHub si no se realizó [push](#).

9. TECNOLOGÍAS UTILIZADAS

El archivo [composer.json](#) del proyecto requiere PHP ^8.2, Laravel ^12.0, Laravel Breeze ^2.4 y PHPUnit ^11.5.50.

El archivo `package.json` incluye Vite, Alpine.js, Tailwind CSS, Axios, PostCSS y el plugin de Laravel para Vite.

9.1. Backend

- PHP 8.2.
- Laravel 12.
- Eloquent ORM.
- Laravel Breeze.
- Form Requests.
- Policies.
- Services.
- Migrations.
- Blade.

9.2. Frontend

- Blade.
- Tailwind CSS.
- Alpine.js.
- Vite.
- JavaScript.
- Axios.

9.3. Datos

- MySQL.
- Migraciones de Laravel.
- Relaciones Eloquent.
- Columnas JSON para configuraciones flexibles.

9.4. Herramientas

- Composer.
- Node.js.
- NPM.
- Git.
- GitHub.
- Visual Studio Code.
- MySQL Workbench.
- PowerShell.

10. ARQUITECTURA DEL SISTEMA

OmniMerge utiliza una arquitectura MVC complementada con Requests, Policies y Services.

Usuario



Ruta



Middleware



Controlador



Form Request / Policy



Modelo o Service



Base de datos



Vista Blade



Respuesta HTML

10.1. Modelo

Representa los datos y relaciones.

Ejemplos:

app/Models/User.php

app/Models/Entity.php

app/Models/Attribute.php

app/Models/Collection.php

10.2. Vista

Representa la interfaz del usuario.

Ejemplos:

resources/views/entities/index.blade.php

resources/views/attributes/show.blade.php

resources/views/community/index.blade.php

10.3. Controlador

Coordina las solicitudes, validación, autorización, consultas y respuestas.

Ejemplos:

app/Http/Controllers/Entities/EntityController.php

app/Http/Controllers/Community/ExploreController.php

10.4. Form Request

Centraliza la validación y autorización de formularios.

Ejemplo:

app/Http/Requests/Entities/StoreEntityRequest.php

10.5. Policy

Determina si un usuario puede ver, crear, editar o eliminar un recurso.

Ejemplo:

app/Policies/EntityPolicy.php

10.6. Service

Contiene lógica compleja que no debe permanecer dentro del controlador.

Ejemplos:

app/Services/Entities/EntityAttributeValueService.php
app/Services/Community/CommunityCloneService.php

11. ESTRUCTURA GENERAL DE CARPETAS

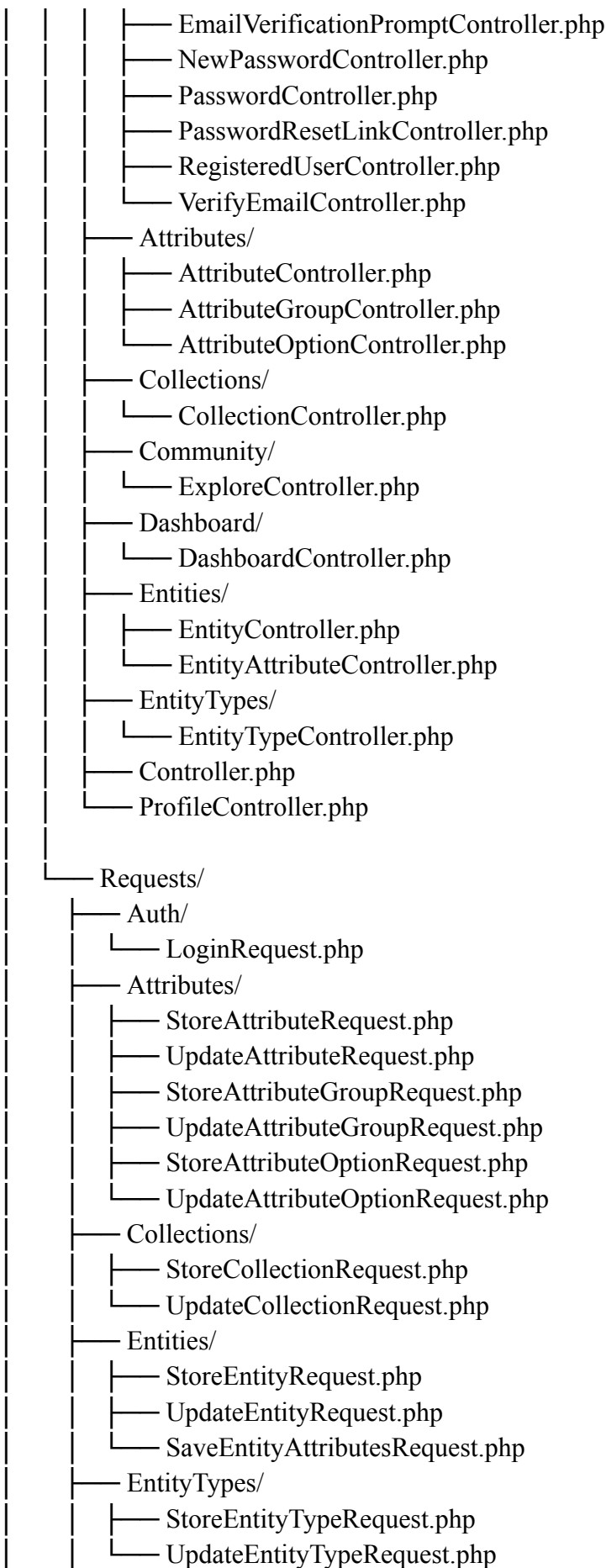
OmniMerge/

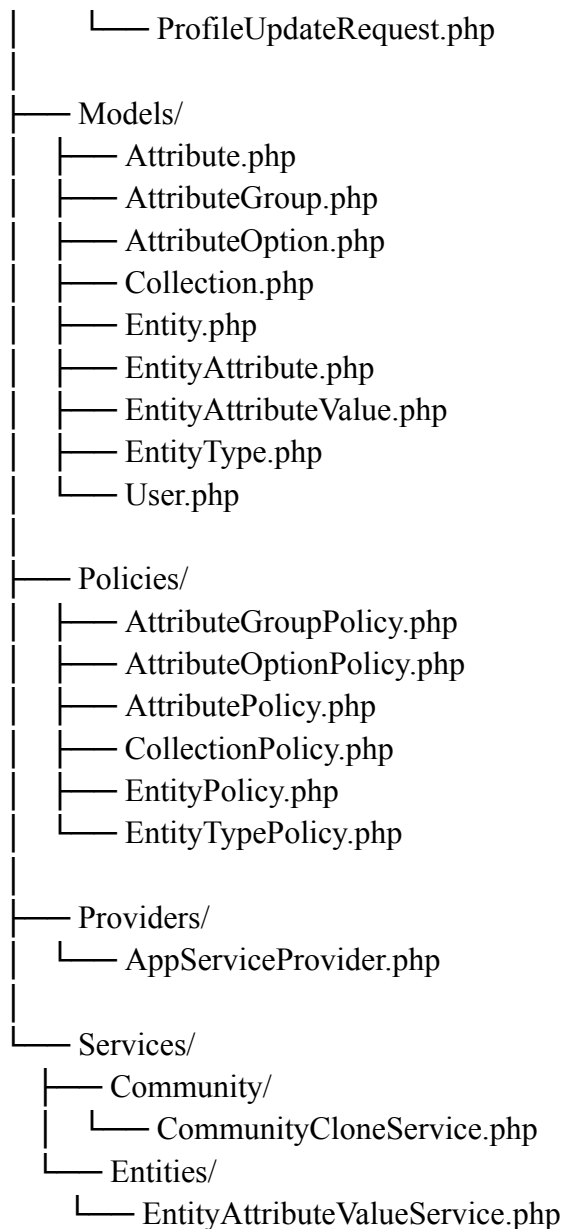
```
|— app/
|   |— Http/
|   |   |— Controllers/
|   |   |— Requests/
|   |— Models/
|   |— Policies/
|   |— Providers/
|   |— Services/
|— bootstrap/
|— config/
|— database/
|   |— factories/
|   |— migrations/
|   |— seeders/
|— public/
|— resources/
|   |— css/
|   |— js/
|   |— views/
|— routes/
|— storage/
|— tests/
|— .env
|— .env.example
|— artisan
|— composer.json
|— package.json
|— vite.config.js
```

12. ESTRUCTURA DETALLADA DE **app**

app/

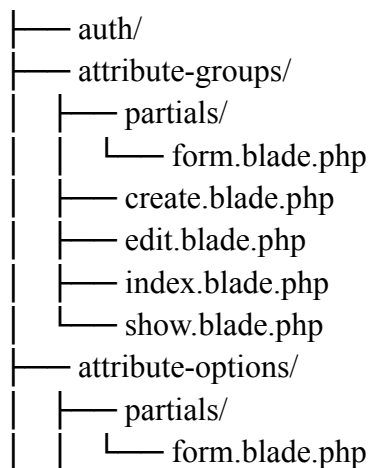
```
|— Http/
|   |— Controllers/
|   |   |— Auth/
|   |   |   |— AuthenticatedSessionController.php
|   |   |   |— ConfirmablePasswordController.php
|   |   |   |— EmailVerificationNotificationController.php
```



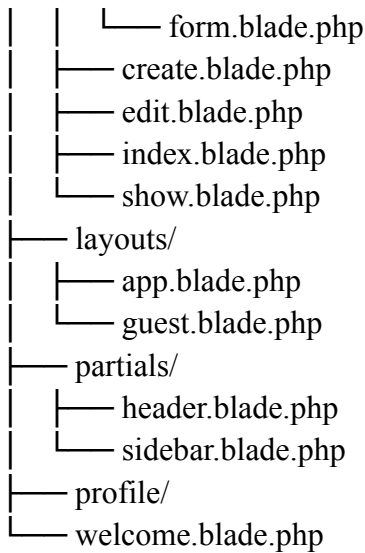


13. ESTRUCTURA DE VISTAS

resources/views/



- create.blade.php
- edit.blade.php
- index.blade.php
- show.blade.php
- attributes/
 - partials/
 - form.blade.php
 - create.blade.php
 - edit.blade.php
 - index.blade.php
 - show.blade.php
- collections/
 - partials/
 - form.blade.php
 - create.blade.php
 - edit.blade.php
 - index.blade.php
 - show.blade.php
- community/
 - partials/
 - attribute-card.blade.php
 - collection-card.blade.php
 - entity-card.blade.php
 - filters.blade.php
 - attribute.blade.php
 - collection.blade.php
 - entity.blade.php
 - index.blade.php
- components/
 - alert.blade.php
 - status-badge.blade.php
 - componentes generados por Breeze
- dashboard/
 - index.blade.php
- entities/
 - partials/
 - form.blade.php
 - attributes.blade.php
 - create.blade.php
 - edit.blade.php
 - index.blade.php
 - show.blade.php
- entity-types/
 - partials/



No son necesarias estas vistas:

attribute-options/partials/create.blade.php

[attribute-options/partials/edit.blade.php](#)

attribute-options/partials/show.blade.php

El único parcial compartido del formulario es:

attribute-options/partials/form.blade.php

Las páginas completas se encuentran directamente en:

attribute-options/create.blade.php

[attribute-options/edit.blade.php](#)

attribute-options/show.blade.php

14. MÓDULO DE AUTENTICACIÓN

14.1. Laravel Breeze

Laravel Breeze fue utilizado como base para:

- Registro.
- Inicio de sesión.
- Cierre de sesión.
- Recuperación de contraseña.
- Confirmación de contraseña.
- Perfil.
- Verificación de correo.

La verificación obligatoria de correo fue retirada temporalmente del dashboard para evitar bloquear al usuario antes de configurar un servidor de correo.

14.2. Tabla **users**

La tabla se amplió con los siguientes campos:

Campo	Descripción
-------	-------------

id	Identificador del usuario
name	Nombre completo
username	Nombre único de usuario
email	Correo electrónico único
email_verified_at	Fecha de verificación
password	Contraseña cifrada
role	Rol del usuario
status	Estado de la cuenta
last_login_at	Último inicio de sesión
remember_token	Token de sesión
created_at	Fecha de creación
updated_at	Fecha de actualización
deleted_at	Borrado lógico

14.3. Roles iniciales

USER

ADMIN

14.4. Estados iniciales

ACTIVE

INACTIVE

SUSPENDED

14.5. Registro

El registro solicita:

- Nombre.
- Nombre de usuario.
- Correo.
- Contraseña.
- Confirmación de contraseña.

Después de registrar al usuario:

1. Se crea el registro.
2. La contraseña se cifra.
3. Se asigna **role = USER**.
4. Se asigna **status = ACTIVE**.
5. Se inicia sesión.

6. Se redirige al dashboard.

14.6. Inicio de sesión

El `LoginRequest` realiza:

1. Control de intentos.
2. Validación de correo y contraseña.
3. Comprobación de cuenta activa.
4. Registro de `last_login_at`.
5. Regeneración de sesión.

14.7. Método `isActive`

Dentro de `User.php`:

```
public function isActive(): bool
{
    return $this->status === 'ACTIVE';
}
```

14.8. Advertencias de `Intelephense`

Cuando `Intelephense` no reconocía:

```
$user->isActive();
$user->forceFill();
```

se solucionó tipando al usuario:

```
use App\Models\User;
```

```
/** @var User $user */
$user = $request->user();
```

15. CONTROLADOR BASE Y AUTORIZACIÓN

Laravel 12 puede generar un controlador base vacío.

Para utilizar:

```
$this->authorize(...)
```

se agregó el trait `AuthorizesRequests`.

Ubicación:

```
app/Http/Controllers/Controller.php
```

Estructura:

```
<?php
```

```
namespace App\Http\Controllers;
```

```
use Illuminate\Foundation\Auth\Access\AuthorizesRequests;
```

```
abstract class Controller
```



```
{  
    use AuthorizesRequests;  
}
```

Esto permite utilizar Políticas desde todos los controladores.

16. DASHBOARD Y LAYOUT PRINCIPAL

16.1. Dashboard

Ubicación:

app/Http/Controllers/Dashboard/DashboardController.php

resources/views/dashboard/index.blade.php

El dashboard muestra:

- Total de tipos de entidad.
- Total de entidades.
- Entidades activas.
- Entidades públicas.
- Entidades recientes.
- Tipos recientes.
- Accesos rápidos.

16.2. Layout

Ubicación:

resources/views/layouts/app.blade.php

El layout:

- Carga Vite.
- Carga Tailwind.
- Carga Alpine.js.
- Incluye el sidebar.
- Incluye el header.
- Incluye alertas.
- Renderiza el contenido mediante `$slot`.

16.3. Sidebar

Ubicación:

resources/views/partials/sidebar.blade.php

Opciones:

- Dashboard.
- Tipos de entidad.
- Entidades.
- Atributos.
- Valores y opciones.
- Grupos de atributos.
- Colecciones.

- Explorar comunidad.
- Universos, futuro.
- Torneos, futuro.
- Rankings, futuro.
- Perfil.

16.4. Header

Ubicación:

resources/views/partials/header.blade.php

Incluye:

- Título de la página.
- Botón de menú móvil.
- Nombre del usuario.
- Rol.
- Acceso al perfil.
- Cierre de sesión.

16.5. Diseño adaptable

Se utilizó Alpine.js para:

- Abrir y cerrar sidebar móvil.
- Mostrar menú del usuario.
- Cerrar desplegables al hacer clic fuera.

17. MÓDULO DE TIPOS DE ENTIDAD

17.1. Propósito

Clasificar entidades mediante categorías personalizadas.

17.2. Modelo

app/Models/EntityType.php

17.3. Controlador

app/Http/Controllers/EntityTypes/EntityTypeController.php

17.4. Requests

app/Http/Requests/EntityTypes/StoreEntityTypeRequest.php

app/Http/Requests/EntityTypes/UpdateEntityTypeRequest.php

17.5. Policy

app/Policies/EntityTypePolicy.php

17.6. Tabla **entity_types**

Campo	Descripción
id	Identificador

<code>user_id</code>	Propietario
<code>code</code>	Código único por usuario
<code>name</code>	Nombre
<code>description</code>	Descripción
<code>icon</code>	Icono
<code>color</code>	Color
<code>status</code>	Estado
<code>sort_order</code>	Orden
<code>created_at</code>	Creación
<code>updated_at</code>	Actualización
<code>deleted_at</code>	Borrado lógico

17.7. Ejemplos

Tipo: Personaje

Código: PERSONAJE

Icono: 

Color: #6366F1

Tipo: País

Código: PAIS

Icono: 

Color: #10B981

17.8. Funciones CRUD

- Listar.
- Buscar.
- Filtrar por estado.
- Crear.
- Mostrar.
- Editar.
- Eliminar.

No se permite eliminar un tipo que tenga entidades asociadas.

18. MÓDULO DE ENTIDADES

18.1. Propósito

Representar cualquier elemento creado por el usuario.

18.2. Modelo

app/Models/Entity.php

18.3. Controlador

app/Http/Controllers/Entities/EntityController.php

18.4. Requests

app/Http/Requests/Entities/StoreEntityRequest.php

app/Http/Requests/Entities/UpdateEntityRequest.php

18.5. Policy

app/Policies/EntityPolicy.php

18.6. Tabla **entities**

Campo	Descripción
id	Identificador
user_id	Propietario
source_entity_id	Entidad original si fue clonada
entity_type_id	Tipo de entidad
code	Código
name	Nombre
slug	Identificador URL
description	Descripción
image	Ruta de imagen
status	Estado
visibility	Visibilidad
allow_cloning	Permitir clonación
views_count	Contador de vistas
clones_count	Contador de copias
published_at	Fecha de publicación
metadata	Datos adicionales JSON
created_at	Creación

updated_at	Actualización
deleted_at	Borrado lógico

18.7. Visibilidad

PRIVATE

PUBLIC

UNLISTED

PRIVATE

Solo el propietario puede acceder.

PUBLIC

Puede aparecer en la comunidad.

UNLISTED

No aparece en búsquedas públicas. En la implementación comunitaria actual también se excluye del detalle público.

18.8. Estados

ACTIVE

INACTIVE

ARCHIVED

18.9. Ejemplo

Nombre: Naruto Uzumaki

Tipo: Personaje

Código: NARUTO_UZUMAKI

Slug: naruto-uzumaki

Estado: ACTIVE

Visibilidad: PUBLIC

18.10. Funciones

- Crear.
 - Editar.
 - Eliminar.
 - Subir imagen.
 - Seleccionar tipo.
 - Definir estado.
 - Definir visibilidad.
 - Buscar.
 - Filtrar.
 - Configurar atributos.
 - Mostrar características.
 - Publicar.
 - Permitir clonación.
-

19. SISTEMA DE IMÁGENES

19.1. Almacenamiento

Las imágenes se almacenan en:

storage/app/public/entities/
storage/app/public/attributes/
storage/app/public/attribute-options/
storage/app/public/collections/

19.2. Enlace público

Se necesita ejecutar:

php artisan storage:link

Esto crea:

public/storage

19.3. Formularios

Todo formulario que envía imágenes debe incluir:

enctype="multipart/form-data"

19.4. Accessor

Los modelos utilizan un accessor:

```
public function getImageUrlAttribute(): ?string
{
    if (! $this->image) {
        return null;
    }

    /** @var \Illuminate\Filesystem\FilesystemAdapter $disk */
    $disk = Storage::disk('public');

    if (! $disk->exists($this->image)) {
        return null;
    }

    return $disk->url($this->image);
}
```

La anotación de `FilesystemAdapter` evita el falso aviso:

Undefined method 'url'

generado por Intelephense.

19.5. Imagen de respaldo

Cuando no existe imagen se muestra:

1. Icono de la entidad o atributo.

2. Icono del tipo.
3. Inicial del nombre.

20. MÓDULO DE ATRIBUTOS DINÁMICOS

20.1. Propósito

Permitir que cada usuario defina características propias.

20.2. Modelo

app/Models/Attribute.php

20.3. Controlador

app/Http/Controllers/Attributes/AttributeController.php

20.4. Requests

app/Http/Requests/Attributes/StoreAttributeRequest.php

app/Http/Requests/Attributes/UpdateAttributeRequest.php

20.5. Policy

app/Policies/AttributePolicy.php

20.6. Tabla **attributes**

Principales campos:

Campo	Descripción
id	Identificador
user_id	Propietario
source_attribute_id	Atributo original del que fue clonado
code	Código
name	Nombre
slug	Identificador URL
description	Descripción
help_text	Ayuda
placeholder	Texto de ejemplo
image	Imagen
icon	Icono
color	Color

data_type	Tipo de dato
value_source	Origen del valor
display_style	Presentación
allows_multiple	Múltiples valores
allows_custom_values	Valores libres adicionales
is_required	Obligatorio
is_filterable	Filtrable
is_comparable	Comparable
is_searchable	Buscable
is_visible	Visible
is_featured	Destacado
min_numeric_value	Mínimo numérico
max_numeric_value	Máximo numérico
min_length	Longitud mínima
max_length	Longitud máxima
unit	Unidad
sort_order	Orden
hierarchy_level	Nivel
scope	Visibilidad
allow_cloning	Permitir copia
views_count	Vistas
clones_count	Copias
published_at	Publicación
default_value	Valor por defecto JSON
validation_rules	Reglas JSON

`configuration`

Configuración JSON

`status`

Estado

20.7. Tipos de dato

TEXT

LONG_TEXT

INTEGER

DECIMAL

BOOLEAN

DATE

COLOR

OPTION

TEXT

Texto corto.

Ejemplo:

Nombre alternativo: El ninja número uno

LONG_TEXT

Texto largo.

Ejemplo:

Historia del personaje

INTEGER

Número entero.

Ejemplo:

Poder: 92

DECIMAL

Número con decimales.

Ejemplo:

Altura: 1.75 m

BOOLEAN

Sí o no.

Ejemplo:

Puede volar: Sí

DATE

Fecha.

Ejemplo:

Fecha de aparición: 1999-09-21

COLOR

Color hexadecimal.

Ejemplo:

Color principal: #FF5733

OPTION

Valor seleccionado desde un catálogo.

Ejemplo:

Anime: Naruto

20.8. Origen de valores

FREE

CATALOG

MIXED

FREE

El usuario escribe el valor.

CATALOG

Debe seleccionar una opción.

MIXED

Puede seleccionar una opción o ingresar un valor personalizado.

20.9. Presentación

TEXTBOX

TEXTAREA

NUMBER

SELECT

MULTISELECT

RADIO

CHECKBOX

TAGS

SLIDER

COLOR_PICKER

DATE_PICKER

20.10. Multiselección

Cuando:

`allows_multiple = true`

una entidad puede poseer varios valores.

Ejemplo:

Elemento:

- Fuego

- Viento

- Luz

Cuando:

`allows_multiple = false`

solo se permite una opción.

Ejemplo:

Anime:

- Naruto

21. OPCIONES DE ATRIBUTOS

21.1. Propósito

Representar valores seleccionables.

Esta tabla reemplaza la idea inicial de crear una tabla separada llamada **adjetivos**.

attribute_options cumple una función más general porque permite:

- Texto.
- Imagen.
- Icono.
- Color.
- Jerarquía.
- Valor numérico.
- Metadatos.
- Estado.

21.2. Modelo

`app/Models/AttributeOption.php`

21.3. Controlador

`app/Http/Controllers/Attributes/AttributeOptionController.php`

21.4. Requests

`app/Http/Requests/Attributes/StoreAttributeOptionRequest.php`

`app/Http/Requests/Attributes/UpdateAttributeOptionRequest.php`

21.5. Policy

`app/Policies/AttributeOptionPolicy.php`

21.6. Tabla **attribute_options**

Campo	Descripción
id	Identificador
attribute_id	Atributo propietario
parent_option_id	Opción superior

code	Código
name	Nombre
description	Descripción
image	Imagen
icon	Icono
color	Color
numeric_value	Valor numérico
sort_order	Orden
metadata	JSON
status	Estado
created_at	Creación
updated_at	Actualización
deleted_at	Borrado lógico

21.7. Ejemplo Anime

Atributo: Anime

Opción 1:

Nombre: Naruto

Código: NARUTO

Imagen: naruto.webp

Icono: 

Color: #F97316

Opción 2:

Nombre: One Piece

Código: ONE_PIECE

Imagen: one-piece.webp

Icono: 

Color: #DC2626

21.8. Jerarquía de opciones

País

└─ Perú

└─ Tacna

└─ Lima

└─ Arequipa

En este caso:

Perú.parent_option_id = null

Tacna.parent_option_id = id de Perú

21.9. Panel propio

El panel se encuentra en:

/attribute-options

Permite:

- Buscar.
- Filtrar por atributo.
- Crear.
- Mostrar.
- Editar.
- Eliminar.
- Subir imagen.
- Seleccionar opción padre.
- Mostrar subopciones.
- Ver cantidad de usos.

22. GRUPOS DE ATRIBUTOS

22.1. Propósito

Organizar visualmente atributos relacionados.

22.2. Modelo

app/Models/AttributeGroup.php

22.3. Tabla **attribute_groups**

Campo	Descripción
id	Identificador
user_id	Propietario
code	Código
name	Nombre
description	Descripción
icon	Icono
color	Color

layout_type	Diseño
collapsible	Puede contraerse
default_expanded	Abierto inicialmente
sort_order	Orden
status	Estado
created_at	Creación
updated_at	Actualización
deleted_at	Borrado lógico

22.4. Diseños

LIST

GRID

CARDS

TABLE

COMPACT

22.5. Relación con atributos

Se utiliza:

attribute_group_attribute

Relación:

Muchos grupos ↔ Muchos atributos

La tabla intermedia contiene:

- attribute_group_id.
- attribute_id.
- custom_label.
- sort_order.
- is_featured.

23. ASIGNACIÓN DE ATRIBUTOS A ENTIDADES

23.1. Tabla **entity_attributes**

Representa que una entidad utiliza un atributo.

Campo	Descripción
id	Identificador
entity_id	Entidad
attribute_id	Atributo

custom_label	Etiqueta personalizada
is_visible	Visible
is_featured	Destacado
sort_order	Orden
notes	Notas
created_at	Creación
updated_at	Actualización

Una entidad no almacena directamente sus atributos en la tabla **entities**.

Ejemplo:

Entidad: Naruto Uzumaki

Atributo: Elemento

genera un registro en **entity_attributes**.

23.2. Tabla **entity_attribute_values**

Guarda los valores concretos.

Campo	Descripción
id	Identificador
entity_attribute_id	Asignación
attribute_option_id	Opción seleccionada
text_value	Texto
integer_value	Entero
decimal_value	Decimal
boolean_value	Booleano
date_value	Fecha
color_value	Color
custom_value	Valor personalizado
json_value	Valor complejo
sort_order	Orden
created_at	Creación

updated_at

Actualización

23.3. Ejemplo completo

Entidad: Naruto Uzumaki

Atributo: Elemento

Permite múltiples valores: Sí

Valores:

- Viento
- Fuego

Base de datos:

entity_attributes

- entity_id: Naruto
- attribute_id: Elemento

entity_attribute_values

- option_id: Viento
- option_id: Fuego

23.4. Service

Ubicación:

app/Services/Entities/EntityAttributeValueService.php

Responsabilidades:

- Crear la asignación.
- Borrar valores anteriores.
- Validar obligatoriedad.
- Validar opciones.
- Guardar valores tipados.
- Manejar múltiples valores.
- Ejecutar el proceso dentro de una transacción.

24. COLECCIONES

24.1. Propósito

Agrupar entidades de acuerdo con un criterio.

24.2. Modelo

app/Models/Collection.php

24.3. Controlador

app/Http/Controllers/Collections/CollectionController.php

24.4. Requests

app/Http/Requests/Collections/StoreCollectionRequest.php

app/Http/Requests/Collections/UpdateCollectionRequest.php

24.5. Policy

app/Policies/CollectionPolicy.php

24.6. Tabla **collections**

Campo	Descripción
id	Identificador
user_id	Propietario
source_collection_id	Colección original
code	Código
name	Nombre
slug	URL
description	Descripción
image	Portada
icon	Icono
color	Color
visibility	Visibilidad
allow_cloning	Permite copia
views_count	Vistas
clones_count	Copias
published_at	Publicación
status	Estado
sort_order	Orden
metadata	JSON
created_at	Creación
updated_at	Actualización
deleted_at	Borrado lógico

24.7. Tabla **collection_entity**

Relación muchos a muchos.

Campo	Descripción
<code>collection_id</code>	Colección
<code>entity_id</code>	Entidad
<code>sort_order</code>	Orden
<code>notes</code>	Notas
<code>added_at</code>	Fecha de incorporación

24.8. Ejemplo

Colección: Protagonistas de anime

Entidades:

1. Naruto Uzumaki
2. Monkey D. Luffy
3. Son Goku

24.9. Funcionalidades

- Crear colección.
- Subir portada.
- Seleccionar icono.
- Seleccionar color.
- Seleccionar entidades.
- Editar.
- Eliminar.
- Definir visibilidad.
- Publicar.
- Permitir clonación.
- Ordenar elementos.

25. EXPLORADOR DE LA COMUNIDAD

25.1. Propósito

Mostrar contenido público de otros usuarios.

Ruta:

/explore

25.2. Controlador

app/Http/Controllers/Community/ExploreController.php

25.3. Vistas

resources/views/community/index.blade.php

resources/views/community/entity.blade.php

resources/views/community/collection.blade.php
resources/views/community/attribute.blade.php

25.4. Contenido explorado

- Entidades.
- Colecciones.
- Atributos.
- Catálogos.

25.5. Filtros

- Búsqueda por nombre.
- Búsqueda por descripción.
- Búsqueda por creador.
- Tipo de entidad.
- Tipo de dato.
- Más populares.
- Más recientes.
- Más antiguos.
- Nombre A–Z.
- Más clonados.
- Más vistos.

25.6. Estadísticas

- Entidades públicas.
- Colecciones públicas.
- Atributos públicos.
- Creadores.

25.7. Restricciones

Solo se muestran registros que cumplan:

Para entidades:

visibility = PUBLIC

status = ACTIVE

published_at no nulo

Para colecciones:

visibility = PUBLIC

status = ACTIVE

published_at no nulo

Para atributos:

scope = PUBLIC

status = ACTIVE

published_at no nulo

26. INTERACCIONES COMUNITARIAS

26.1. Tabla **community_interactions**

Campo	Descripción
id	Identificador
user_id	Usuario
content_type	Tipo
content_id	Recurso
interaction_type	Interacción
metadata	Información extra
created_at	Creación
updated_at	Actualización

26.2. Tipos de contenido

ENTITY

COLLECTION

ATTRIBUTE

26.3. Tipos de interacción

VIEW

CLONE

FAVORITE

FAVORITE está preparado conceptualmente, pero todavía falta implementar su interfaz y lógica.

27. CLONACIÓN COMUNITARIA

27.1. Service

app/Services/Community/CommunityCloneService.php

27.2. Principio de clonación

No se comparte el mismo registro.

Se crea una copia privada para el usuario.

Esto evita que:

- El creador original modifique la copia.
- El usuario que copia afecte al original.
- Dos usuarios compartan accidentalmente los mismos datos editables.

27.3. Clonar una entidad

El servicio puede copiar:

- Entidad.
- Tipo de entidad.

- Imagen.
- Atributos.
- Opciones.
- Valores.
- Metadatos.

La copia queda inicialmente:

visibility = PRIVATE

status = ACTIVE

27.4. Clonar un atributo

Copia:

- Configuración.
- Tipo de dato.
- Imagen.
- Icono.
- Color.
- Opciones.
- Jerarquía de opciones.
- Reglas.
- Unidades.
- Presentación.

27.5. Clonar una colección

Copia:

- Colección.
- Portada.
- Configuración.
- Entidades públicas y clonables.
- Relaciones de orden.

27.6. Procedencia

Se registra mediante:

source_entity_id

source_collection_id

source_attribute_id

27.7. Consideración de diseño

En el modelo actual, `source_attribute_id` se utiliza como referencia al atributo original del cual fue clonada una copia.

Las futuras dependencias entre atributos no deben utilizar esa misma columna.

Para las jerarquías y condiciones se utilizarán tablas independientes:

attribute_relationships

attribute_conditions

attribute_option_relationships

Esto evita mezclar:

- Procedencia de clonación.
 - Dependencia funcional.
 - Jerarquía visual.
 - Condiciones de visibilidad.
-

28. RELACIONES PRINCIPALES

User

- ├── hasMany EntityType
- ├── hasMany Entity
- ├── hasMany Attribute
- ├── hasMany AttributeGroup
- └── hasMany Collection

EntityType

- ├── belongsTo User
- └── hasMany Entity

Entity

- ├── belongsTo User
- ├── belongsTo EntityType
- ├── hasMany EntityAttribute
- ├── belongsToMany Collection
- ├── belongsTo Entity como origen
- └── hasMany Entity como copias

Attribute

- ├── belongsTo User
- ├── hasMany AttributeOption
- ├── hasMany EntityAttribute
- ├── belongsToMany AttributeGroup
- ├── belongsTo Attribute como origen
- └── hasMany Attribute como copias

AttributeOption

- ├── belongsTo Attribute
- ├── belongsTo AttributeOption como padre
- ├── hasMany AttributeOption como hijos
- └── hasMany EntityAttributeValue

AttributeGroup

- ├── belongsTo User
- └── belongsToMany Attribute

EntityAttribute

- └ belongsTo Entity
- └ belongsTo Attribute
- └ hasMany EntityAttributeValue

EntityAttributeValue

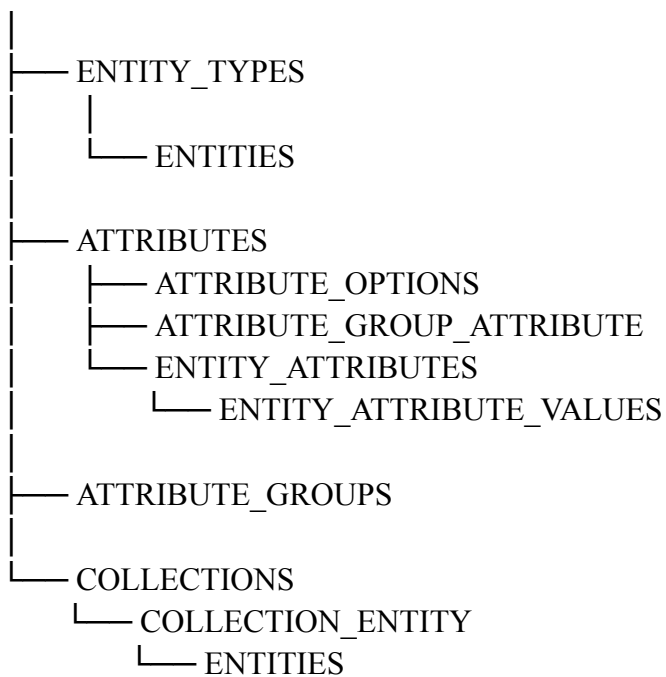
- └ belongsTo EntityAttribute
- └ belongsTo AttributeOption

Collection

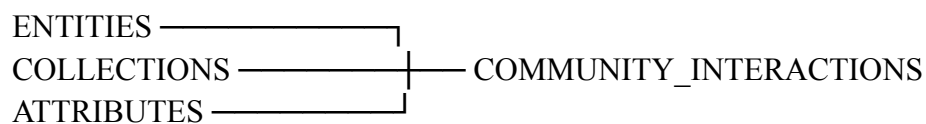
- └ belongsTo User
- └ belongsToMany Entity
- └ belongsTo Collection como origen
- └ hasMany Collection como copias

29. DIAGRAMA LÓGICO SIMPLIFICADO

USERS



Comunidad:



30. RUTAS PRINCIPALES

30.1. Autenticación

GET /register

POST /register

GET /login
POST /login
POST /logout

30.2. Dashboard

GET /dashboard

30.3. Perfil

GET /profile
PATCH /profile
DELETE /profile

30.4. Tipos de entidad

GET /entity-types
GET /entity-types/create
POST /entity-types
GET /entity-types/{entity_type}
GET /entity-types/{entity_type}/edit
PUT /entity-types/{entity_type}
DELETE /entity-types/{entity_type}

30.5. Entidades

GET /entities
GET /entities/create
POST /entities
GET /entities/{entity}
GET /entities/{entity}/edit
PUT /entities/{entity}
DELETE /entities/{entity}

30.6. Atributos de entidad

GET /entities/{entity}/attributes
PUT /entities/{entity}/attributes

30.7. Atributos

GET /attributes
GET /attributes/create
POST /attributes
GET /attributes/{attribute}
GET /attributes/{attribute}/edit
PUT /attributes/{attribute}
DELETE /attributes/{attribute}

30.8. Opciones

GET /attribute-options
GET /attribute-options/create
GET /attribute-options/{attributeOption}
GET /attribute-options/{attributeOption}/edit
POST /attributes/{attribute}/options
PUT /attributes/{attribute}/options/{option}
DELETE /attribute-options/{attributeOption}

30.9. Grupos

GET /attribute-groups
GET /attribute-groups/create
POST /attribute-groups
GET /attribute-groups/{attribute_group}
GET /attribute-groups/{attribute_group}/edit
PUT /attribute-groups/{attribute_group}
DELETE /attribute-groups/{attribute_group}

30.10. Colecciones

GET /collections
GET /collections/create
POST /collections
GET /collections/{collection}
GET /collections/{collection}/edit
PUT /collections/{collection}
DELETE /collections/{collection}

30.11. Comunidad

GET /explore
GET /explore/entities/{entity}
GET /explore/collections/{collection}
GET /explore/attributes/{attribute}

POST /explore/entities/{entity}/clone
POST /explore/collections/{collection}/clone
POST /explore/attributes/{attribute}/clone

31. SEGURIDAD

31.1. Middleware

Las rutas internas utilizan:
Route::middleware('auth')

31.2. Policies

Cada usuario solo puede editar y eliminar sus propios registros.

Ejemplo conceptual:

```
return $entity->user_id === $user->id;
```

31.3. Contenido público

La comunidad no utiliza consultas sin filtros.

No debe utilizarse:

```
Entity::all();
```

Debe utilizarse:

```
Entity::query()
```

```
->where('visibility', 'PUBLIC')
```

```
->where('status', 'ACTIVE')
```

```
->whereNotNull('published_at');
```

31.4. Validación de pertenencia

Las reglas **exists** se restringen por **user_id**.

Ejemplo:

```
Rule::exists('entities', 'id')
```

```
->where(
```

```
    fn ($query) => $query
```

```
        ->where('user_id', $this->user()->id)
```

```
        ->whereNull('deleted_at')
```

```
)
```

Esto evita que un usuario envíe manualmente el ID de una entidad ajena.

31.5. CSRF

Los formularios utilizan:

```
@csrf
```

31.6. Métodos HTTP

```
@method('PUT')
```

```
@method('DELETE')
```

31.7. Contraseñas

El modelo **User** utiliza:

```
'password' => 'hashed'
```

La contraseña nunca debe almacenarse como texto plano.

32. VALIDACIONES

32.1. Códigos

Los códigos se normalizan con:

```
Str::upper(
```

```
    Str::slug($value, '_')
```

)

Ejemplo:

Naruto Uzumaki

→ NARUTO_UZUMAKI

32.2. Slugs

Str::slug(\$name)

Ejemplo:

Naruto Uzumaki

→ naruto-uzumaki

32.3. Imágenes

Formatos:

jpg

jpeg

png

webp

Tamaños utilizados:

- Entidad: aproximadamente 2 MB.
- Opción: aproximadamente 3 MB.
- Colección: aproximadamente 4 MB.

32.4. Valores únicos

El código y slug son únicos por usuario, no necesariamente globales.

Esto permite que dos usuarios tengan:

NARUTO

sin entrar en conflicto.

33. FLUJOS DE USO

33.1. Crear una entidad sencilla

1. Iniciar sesión.
2. Abrir [Tipos de entidad](#).
3. Crear [Personaje](#).
4. Abrir [Entidades](#).
5. Seleccionar [Nueva entidad](#).
6. Escribir [Naruto Uzumaki](#).
7. Seleccionar [Personaje](#).
8. Subir imagen.
9. Guardar.

33.2. Crear un atributo seleccionable

1. Abrir [Atributos](#).

2. Crear atributo.
3. Nombre: **Anime**.
4. Tipo: **OPTION**.
5. Origen: **CATALOG**.
6. Presentación: **SELECT** o **MULTISELECT**.
7. Guardar.

33.3. Crear opciones

1. Abrir **Valores y opciones**.
2. Seleccionar atributo **Anime**.
3. Crear **Naruto**.
4. Añadir imagen.
5. Crear **One Piece**.
6. Añadir imagen.
7. Crear **Dragon Ball**.
8. Añadir imagen.

33.4. Asignar atributos

1. Abrir una entidad.
2. Seleccionar **Configurar atributos**.
3. Seleccionar **Anime = Naruto**.
4. Seleccionar **Elemento = Viento**.
5. Escribir **Poder = 92**.
6. Guardar.

33.5. Crear una colección

1. Abrir **Colecciones**.
2. Crear **Protagonistas de anime**.
3. Subir portada.
4. Seleccionar Naruto.
5. Seleccionar Luffy.
6. Seleccionar Goku.
7. Guardar.

33.6. Publicar

1. Editar entidad.
2. Cambiar visibilidad a **PUBLIC**.
3. Activar **Permitir clonación**.
4. Guardar.
5. Se asigna **published_at**.

33.7. Clonar contenido

1. Ingresar con otro usuario.
2. Abrir **Explorar comunidad**.
3. Buscar **Naruto**.
4. Abrir el detalle.
5. Seleccionar **Copiar a mi biblioteca**.
6. El sistema crea una copia privada.
7. El usuario puede editarla sin afectar al original.

34. INSTALACIÓN DEL PROYECTO

34.1. Requisitos

- PHP 8.2 o superior.
- Composer.
- Node.js.
- NPM.
- MySQL.
- Git.

34.2. Clonar

```
git clone <URL_DEL_REPOSITORIO>
cd OmniMerge
```

34.3. Dependencias PHP

```
composer install
```

34.4. Dependencias frontend

En PowerShell puede utilizarse:
`npm.cmd install`

En CMD o Bash:
`npm install`

34.5. Archivo de entorno

```
copy .env.example .env
```

En Linux:
`cp .env.example .env`

34.6. Clave

```
php artisan key:generate
```

34.7. Base de datos

Crear:
`CREATE DATABASE omnimerge`
`CHARACTER SET utf8mb4`
`COLLATE utf8mb4_unicode_ci;`

34.8. Configurar **.env**

```
APP_NAME=OmniMerge
APP_ENV=local
APP_DEBUG=true
APP_URL=http://127.0.0.1:8000
```

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=omnimerge
DB_USERNAME=root
DB_PASSWORD=
```

34.9. Migraciones

```
php artisan migrate
```

Para reconstruir completamente:

```
php artisan migrate:fresh
```

Advertencia: **migrate:fresh** elimina todos los datos.

34.10. Storage

```
php artisan storage:link
```

34.11. Compilar

```
npm.cmd run build
```

34.12. Ejecutar

Terminal 1:

```
php artisan serve
```

Terminal 2:

```
npm.cmd run dev
```

Abrir:

```
http://127.0.0.1:8000
```

35. COMANDOS DE DESARROLLO

Limpiar caché

```
php artisan optimize:clear
```

```
php artisan view:clear
```

```
php artisan route:clear
```

```
php artisan config:clear
```

Ver rutas

```
php artisan route:list
```

Ver migraciones

```
php artisan migrate:status
```

Ejecutar tests

php artisan test

Abrir Tinker

php artisan tinker

Comprobar vista

view()->exists('collections.index');

36. PRUEBAS

36.1. Problema encontrado

Los tests de Breeze fallaron inicialmente con:

NOT NULL constraint failed: users.username

La causa fue que **UserFactory** no generaba **username**.

36.2. Solución

Ubicación:

database/factories/UserFactory.php

Debe incluir:

```
'username' => fake()->unique()->userName(),  
'role' => 'USER',  
'status' => 'ACTIVE',
```

36.3. RegistrationTest

Los tests de registro deben enviar:

```
'username' => 'usuario_prueba',
```

36.4. Pruebas recomendadas

- Usuario puede registrarse.
- Usuario puede iniciar sesión.
- Usuario inactivo no puede iniciar sesión.
- Usuario puede crear entidad.
- Usuario no puede editar entidad ajena.
- Usuario puede crear atributo.
- Multiselección guarda varios valores.
- Imagen se almacena.
- Colección relaciona entidades.
- Contenido privado no aparece en comunidad.
- Contenido público aparece.
- Clonación genera un registro independiente.

37. ERRORES FRECUENTES

37.1. Vista no encontrada

Error:

View [collections.index] not found

Solución:

Crear:

resources/views/collections/index.blade.php

Limpiar:

php artisan view:clear

php artisan optimize:clear

37.2. Undefined method authorize

Agregar:

```
use Illuminate\Foundation\Auth\Access\AuthorizesRequests;
```

```
abstract class Controller
```

```
{
```

```
    use AuthorizesRequests;
```

```
}
```

37.3. Undefined method url

Tipar disco:

```
/** @var \Illuminate\Filesystem\FilesystemAdapter $disk */
```

```
$disk = Storage::disk('public');
```

37.4. Imagen no aparece

Comprobar:

```
php artisan storage:link
```

Comprobar:

```
storage/app/public/
```

```
public/storage
```

Comprobar `enctype`.

37.5. Undefined method user o id

Usar:

```
use App\Models\User;
```

```
use Illuminate\Http\Request;
```

```
/** @var User $user */
```

```
$user = $request->user();
```

```
$user->id;
```


37.6. Error de NPM en PowerShell

Usar:

npm.cmd install

npm.cmd run dev

37.7. Tabla ya existe

Durante desarrollo:

php artisan migrate:fresh

No crear manualmente las mismas tablas que Laravel administra.

38. GIT Y GITHUB

Estado

git status

Añadir

git add .

Revisar

git diff --cached

Commit

git commit -m "feat: describir cambio"

Push

git push origin main

Ejemplos de commits

feat(auth): configurar registro e inicio de sesión

feat(core): implementar dashboard y CRUD de entidades

feat(attributes): implementar atributos dinámicos

feat(collections): agregar colecciones y opciones visuales

feat(community): implementar explorador y clonación

fix(storage): corregir visualización de imágenes

docs: documentar arquitectura de OmniMerge

39. ESTADO POR SPRINT

Sprint 0: Base

- Laravel.
- MySQL.
- GitHub.
- Breeze.

- Login.
- Registro.
- Perfil.

Sprint 1: Biblioteca básica

- Dashboard.
- Layout.
- Sidebar.
- Tipos de entidad.
- Entidades.
- Imágenes.
- Políticas.

Sprint 2: Atributos dinámicos

- Atributos.
- Tipos de dato.
- Opciones.
- Multiselección.
- Valores tipados.
- Grupos.
- Asignación a entidades.

Sprint 3: Organización

- Panel de opciones.
- Imágenes en opciones.
- Colecciones.
- Portadas.
- Relación colección-entidad.

Sprint 4: Comunidad

- Explorador.
- Filtros.
- Publicación.
- Estadísticas.
- Vistas.
- Clonación.
- Procedencia.

40. DEUDA TÉCNICA Y MEJORAS RECOMENDADAS

40.1. Completar carga de imagen de atributo

La base de datos y modelo contemplan `attributes.image`.

Debe verificarse que:

- El formulario tenga `enctype`.
- Los Requests validen `image`.
- `AttributeController` almacene la imagen.
- Update permita eliminarla.
- Destroy elimine el archivo.

40.2. Renombrar modelos potencialmente ambiguos

Los nombres:

App\Models\Attribute

App\Models\Collection

pueden confundirse con:

\Attribute

Illuminate\Support\Collection

Es posible mantenerlos usando imports explícitos, pero una futura refactorización podría considerar:

DynamicAttribute

EntityCollection

No es obligatorio cambiarlo actualmente.

40.3. Condiciones

Crear:

attribute_relationships

attribute_conditions

attribute_option_relationships

40.4. Tipos globales

Actualmente cada usuario crea tipos propios.

Para una comunidad más consistente se recomienda:

entity_type_categories

Ejemplos globales:

- Personaje.
- Lugar.
- Objeto.
- Criatura.
- Concepto.

40.5. Favoritos

Implementar:

community_interactions.interaction_type = FAVORITE

40.6. Control de vistas

Evitar contar repetidamente cada recarga del mismo usuario o IP.

40.7. Moderación

Añadir:

- Reportar contenido.
- Ocultar.
- Suspender.
- Revisar.

- Estado de moderación.

40.8. Clonación de colección

Debe copiar únicamente entidades públicas y permitidas.

40.9. Transacciones

Mantener clonaciones y guardado de valores dentro de transacciones.

40.10. Tests

Completar tests Feature para todos los módulos.

41. ROADMAP FUTURO

Fase 1: Biblioteca

Estado: desarrollado en gran parte.

- Usuarios.
- Entidades.
- Atributos.
- Opciones.
- Colecciones.
- Comunidad.

Fase 2: Universos

Tablas propuestas:

universes

universe_entities

seasons

season_entities

Funciones:

- Crear universo.
- Añadir entidades.
- Definir reglas.
- Crear temporadas.
- Registrar estado temporal.

Fase 3: Torneos

Tablas propuestas:

tournaments

tournament_participants

tournament_rounds

tournament_matches

match_results

Funciones:

- Seleccionar participantes.
- Filtrar por atributos.
- Generar llaves.
- Simular encuentros.
- Registrar ganadores.

Fase 4: Motor de simulación

- Fórmulas.
- Pesos.
- Probabilidad.
- Ventajas.
- Desventajas.
- Aleatoriedad controlada.
- Eventos.

Fase 5: Narrativa

- Descripción automática de eventos.
- Historial.
- Temporadas.
- Rivalidades.
- Logros.
- Evolución.

Fase 6: Analítica

- Rankings.
- Comparaciones.
- Tendencias.
- Entidades más exitosas.
- Atributos más influyentes.
- Gráficos.

42. GLOSARIO

Entidad

Elemento concreto creado por un usuario.

Tipo de entidad

Clasificación de una entidad.

Atributo

Característica reutilizable.

Opción

Valor seleccionable de un atributo.

Grupo de atributos

Agrupación visual.

Colección

Agrupación reutilizable de entidades.

Catálogo

Conjunto de opciones.

Multiselección

Posibilidad de elegir más de una opción.

Scope

Visibilidad de un atributo.

Visibility

Visibilidad de entidad o colección.

Clonar

Crear una copia independiente.

Slug

Texto utilizado en una URL.

Policy

Regla de autorización.

Request

Clase de validación.

Service

Clase con lógica de negocio compleja.

Soft Delete

Borrado lógico mediante `deleted_at`.

43. CONCLUSIÓN

OmniMerge ha evolucionado desde una idea de creación de personajes hacia una plataforma general de modelado de entidades.

La arquitectura implementada permite que cada usuario:

- Cree sus propios tipos.
- Cree cualquier entidad.
- Defina atributos.
- Cree catálogos visuales.
- Asigne múltiples valores.
- Organice atributos.
- Organice entidades.
- Publique contenido.
- Explore contenido comunitario.
- Clone contenido sin alterar el original.

La estructura actual constituye una base sólida para desarrollar los módulos más complejos del proyecto:

- Universos.
- Temporadas.
- Torneos.
- Simulaciones.
- Historial.
- Rankings.
- Narrativa emergente.

El principio central de OmniMerge es mantener la máxima flexibilidad posible: el sistema no decide qué debe crear el usuario, sino que proporciona las herramientas para que el usuario construya su propia estructura.